Assignment 7

Part 1: Query Generation

Query 1 — Aggregation

Prompt:

"Given the following PostgreSQL database schema: create table classroom 2 (building varchar(15), 3 room_number varchar(7), 4 capacity numeric(4,0), 5 primary key (building, room_number) 6); 7 8 create table department 9 (dept_name varchar(20), 10 building varchar(15), 11 budget numeric(12,2) check (budget > 0), 12 primary key (dept_name) 13); 14 15 create table course 16 (course_id varchar(8), 17 title varchar(50), 18 dept_name varchar(20), 19 credits numeric(2,0) check (credits > 0), 20 primary key (course_id), 21 foreign key (dept_name) references department (dept_name) 22 on delete set null 23); 24 25 create table instructor 26 (ID varchar(5), 27 name varchar(20) not null, 28 dept_name varchar(20), 29 salary numeric(8,2) check (salary > 29000), 30 primary key (ID), 31 foreign key (dept_name) references department (dept_name) 32 on delete set null 33); 34 35 create table section 36 (course_id varchar(8), 37 sec_id varchar(8), 38 semester varchar(6) 39 check (semester in ('Fall', 'Winter', 'Spring', 'Summer')), 40 year numeric(4,0) check (year > 1701 and year < 2100), 41 building varchar(15), 42 room_number varchar(7), 43 time_slot_id varchar(4), 44 primary key (course_id, sec_id, semester, year), 45 foreign key (course_id) references course (course_id) 46 on delete cascade, 47 foreign key (building, room_number) 48 references classroom (building, room_number) 49 on delete set null 50); 51 52 create table teaches 53 (ID varchar(5), 54 course_id varchar(8), 55 sec_id varchar(8), 56 semester varchar(6), 57 year numeric(4,0), 58 primary key (ID, course_id, sec_id, semester, year), 59 foreign key (course_id, sec_id, semester, year) 60 references section (course_id, sec_id, semester, year) 61 on delete cascade, 62 foreign key (ID) references instructor (ID) 63 on delete cascade 64); 65 66 create table student 67 (ID varchar(5), 68 name varchar(20) not null, 69 dept_name varchar(20), 70 tot_cred numeric(3,0) check (tot_cred >= 0), 71 primary key (ID), 72 foreign key (dept_name) references department (dept_name) 73 on delete set null 74); 75 76 create table takes 77 (ID varchar(5), 78 course_id varchar(8), 79 sec_id varchar(8), 80 semester varchar(6), 81 year numeric(4,0), 82 grade varchar(2), 83 primary key (ID, course_id, sec_id, semester, year), 84 foreign key (course_id, sec_id, semester, year) 85 references section (course_id, sec_id,

semester, year) 86 on delete cascade, 87 foreign key (ID) references student (ID) 88 on delete cascade 89); 90 91 create table advisor 92 (s_ID varchar(5), 93 i_ID varchar(5), 94 primary key (s_ID), 95 foreign key (i_ID) references instructor (ID) 96 on delete set null, 97 foreign key (s_ID) references student (ID) 98 on delete cascade 99); 100 101 create table time_slot 102 (time_slot_id varchar(4), 103 day varchar(1), 104 start_hr numeric(2) check (start_hr >= 0 and start_hr < 24), 105 start_min numeric(2) check (start_min >= 0 and start_min < 60), 106 end_hr numeric(2) check (end_hr >= 0 and end_hr < 24), 107 end_min numeric(2) check (end_min >= 0 and end_min < 60), 108 primary key (time_slot_id, day, start_hr, start_min) 109); 110 111 create table prereq 112 (course_id varchar(8), 113 prereq_id varchar(8), 114 primary key (course_id, prereq_id), 115 foreign key (course_id) references course (course_id) 116 on delete cascade, 117 foreign key (prereq_id) references course (course_id) 118);

The database contains approximately: - 2,000 students - 30,000 enrollment records in "takes" - 200 courses - 100 sections - 50 instructors - 20 departments

Write a SQL query to: Find the total number of course enrollments and the average total credits (tot_cred) of students in each department. Rank departments from highest enrollment count to lowest. Please use standard PostgreSQL syntax."

AI-Generated SQL:

```
SELECT
    s.dept_name,
    COUNT(*) AS total_enrollments,
    AVG(s.tot_cred) AS avg_total_credits
FROM student AS s
JOIN takes AS t
    ON s.ID = t.ID
GROUP BY s.dept_name
ORDER BY total_enrollments DESC;
```

Note:

Used ChatGPT 5.3 March 31, 2026

Query 2 — Multi-Table Join

Prompt:

Given the following PostgreSQL database schema: create table classroom 2 (building varchar(15), 3 room_number varchar(7), 4 capacity numeric(4,0), 5 primary key (building, room_number) 6); 7 8 create table department 9 (dept_name varchar(20), 10 building varchar(15), 11 budget

numeric(12,2) check (budget > 0), 12 primary key (dept_name) 13); 14 15 create table course 16 (course_id varchar(8), 17 title varchar(50), 18 dept_name varchar(20), 19 credits numeric(2,0) check (credits > 0), 20 primary key (course_id), 21 foreign key (dept_name) references department (dept_name) 22 on delete set null 23); 24 25 create table instructor 26 (ID varchar(5), 27 name varchar(20) not null, 28 dept_name varchar(20), 29 salary numeric(8,2) check (salary > 29000), 30 primary key (ID), 31 foreign key (dept_name) references department (dept_name) 32 on delete set null 33); 34 35 create table section 36 (course_id varchar(8), 37 sec_id varchar(8), 38 semester varchar(6) 39 check (semester in ('Fall', 'Winter', 'Spring', 'Summer')), 40 year numeric(4,0) check (year > 1701 and year < 2100), 41 building varchar(15), 42 room_number varchar(7), 43 time_slot_id varchar(4), 44 primary key (course_id, sec_id, semester, year), 45 foreign key (course_id) references course (course_id) 46 on delete cascade, 47 foreign key (building, room_number) 48 references classroom (building, room_number) 49 on delete set null 50); 51 52 create table teaches 53 (ID varchar(5), 54 course_id varchar(8), 55 sec_id varchar(8), 56 semester varchar(6), 57 year numeric(4,0), 58 primary key (ID, course_id, sec_id, semester, year), 59 foreign key (course_id, sec_id, semester, year) 60 references section (course_id, sec_id, semester, year) 61 on delete cascade, 62 foreign key (ID) references instructor (ID) 63 on delete cascade 64); 65 66 create table student 67 (ID varchar(5), 68 name varchar(20) not null, 69 dept_name varchar(20), 70 tot_cred numeric(3,0) check (tot_cred >= 0), 71 primary key (ID), 72 foreign key (dept_name) references department (dept_name) 73 on delete set null 74); 75 76 create table takes 77 (ID varchar(5), 78 course_id varchar(8), 79 sec_id varchar(8), 80 semester varchar(6), 81 year numeric(4,0), 82 grade varchar(2), 83 primary key (ID, course_id, sec_id, semester, year), 84 foreign key (course_id, sec_id, semester, year) 85 references section (course_id, sec_id, semester, year) 86 on delete cascade, 87 foreign key (ID) references student (ID) 88 on delete cascade 89); 90 91 create table advisor 92 (s_ID varchar(5), 93 i_ID varchar(5), 94 primary key (s_ID), 95 foreign key (i_ID) references instructor (ID) 96 on delete set null, 97 foreign key (s_ID) references student (ID) 98 on delete cascade 99); 100 101 create table time_slot 102 (time_slot_id varchar(4), 103 day varchar(1), 104 start_hr numeric(2) check (start_hr >= 0 and start_hr < 24), 105 start_min numeric(2) check (start_min >= 0 and start_min < 60), 106 end_hr numeric(2) check (end_hr >= 0 and end_hr < 24), 107 end_min numeric(2) check (end_min >= 0 and end_min < 60), 108 primary key (time_slot_id, day, start_hr, start_min) 109); 110 111 create table prereq 112 (course_id varchar(8), 113 prereq_id varchar(8), 114 primary key (course_id, prereq_id), 115 foreign key (course_id) references course (course_id) 116 on delete cascade, 117 foreign key (prereq_id) references course (course_id) 118);

The database contains approximately: - 2,000 students - 30,000 enrollment records in "takes" - 200 courses - 100 sections - 50 instructors - 20 departments

Write a SQL query to: For each instructor, list their name, department, the courses they teach, the semester and year, and the number of students enrolled in each of their sections. Include only sections with at least 10 students. Please use standard PostgreSQL syntax.

AI-Generated SQL:

```
SELECT
    i.name AS instructor_name,
    i.dept_name AS instructor_department,
    c.course_id,
    c.title AS course_title,
    s.sec_id,
    s.semester,
    s.year,
    COUNT(tk.ID) AS enrolled_students
FROM instructor AS i
JOIN teaches AS te
    ON i.ID = te.ID
JOIN section AS s
    ON te.course_id = s.course_id
   AND te.sec_id = s.sec_id
   AND te.semester = s.semester
   AND te.year = s.year
JOIN course AS c
    ON s.course_id = c.course_id
JOIN takes AS tk
    ON s.course_id = tk.course_id
   AND s.sec_id = tk.sec_id
   AND s.semester = tk.semester
   AND s.year = tk.year
GROUP BY
    i.ID,
    i.name,
    i.dept_name,
    c.course_id,
    c.title,
    s.sec_id,
    s.semester,
    s.year
HAVING COUNT(tk.ID) >= 10
ORDER BY
    i.name,
    s.year,
    s.semester,
    c.course_id,
    s.sec_id;
```

Note:

Used ChatGPT 5.3 April 1, 2026

Query 3 — Correlated Subquery

Prompt:

Given the following PostgreSQL database schema: create table classroom 2 (building varchar(15), 3 room_number varchar(7), 4 capacity numeric(4,0), 5 primary key (building, room_number) 6); 7 8 create table department 9 (dept_name varchar(20), 10 building varchar(15), 11 budget numeric(12,2) check (budget > 0), 12 primary key (dept_name) 13); 14 15 create table course 16 (course_id varchar(8), 17 title varchar(50), 18 dept_name varchar(20), 19 credits numeric(2,0) check (credits > 0), 20 primary key (course_id), 21 foreign key (dept_name) references department (dept_name) 22 on delete set null 23); 24 25 create table instructor 26 (ID varchar(5), 27 name varchar(20) not null, 28 dept_name varchar(20), 29 salary numeric(8,2) check (salary > 29000), 30 primary key (ID), 31 foreign key (dept_name) references department (dept_name) 32 on delete set null 33); 34 35 create table section 36 (course_id varchar(8), 37 sec_id varchar(8), 38 semester varchar(6) 39 check (semester in ('Fall', 'Winter', 'Spring', 'Summer')), 40 year numeric(4,0) check (year > 1701 and year < 2100), 41 building varchar(15), 42 room_number varchar(7), 43 time_slot_id varchar(4), 44 primary key (course_id, sec_id, semester, year), 45 foreign key (course_id) references course (course_id) 46 on delete cascade, 47 foreign key (building, room_number) 48 references classroom (building, room_number) 49 on delete set null 50); 51 52 create table teaches 53 (ID varchar(5), 54 course_id varchar(8), 55 sec_id varchar(8), 56 semester varchar(6), 57 year numeric(4,0), 58 primary key (ID, course_id, sec_id, semester, year), 59 foreign key (course_id, sec_id, semester, year) 60 references section (course_id, sec_id, semester, year) 61 on delete cascade, 62 foreign key (ID) references instructor (ID) 63 on delete cascade 64); 65 66 create table student 67 (ID varchar(5), 68 name varchar(20) not null, 69 dept_name varchar(20), 70 tot_cred numeric(3,0) check (tot_cred >= 0), 71 primary key (ID), 72 foreign key (dept_name) references department (dept_name) 73 on delete set null 74); 75 76 create table takes 77 (ID varchar(5), 78 course_id varchar(8), 79 sec_id varchar(8), 80 semester varchar(6), 81 year numeric(4,0), 82 grade varchar(2), 83 primary key (ID, course_id, sec_id, semester, year), 84 foreign key (course_id, sec_id, semester, year) 85 references section (course_id, sec_id, semester, year) 86 on delete cascade, 87 foreign key (ID) references student (ID) 88 on delete cascade 89); 90 91 create table advisor 92 (s_ID varchar(5), 93 i_ID varchar(5), 94 primary key (s_ID), 95 foreign key (i_ID) references instructor (ID) 96 on delete set null, 97 foreign key (s_ID) references student (ID) 98 on delete cascade 99); 100 101 create table time_slot 102 (time_slot_id varchar(4), 103 day varchar(1), 104 start_hr numeric(2) check (start_hr >= 0 and start_hr < 24), 105 start_min numeric(2) check (start_min >= 0 and start_min < 60), 106 end_hr numeric(2) check (end_hr >= 0 and end_hr < 24), 107 end_min numeric(2) check (end_min >= 0 and end_min < 60), 108 primary key (time_slot_id, day, start_hr, start_min) 109); 110 111 create table prereq 112 (course_id varchar(8), 113 prereq_id varchar(8), 114 primary key (course_id, prereq_id), 115

foreign key (course_id) references course (course_id) 116 on delete cascade, 117 foreign key (prereq_id) references course (course_id) 118);

The database contains approximately: - 2,000 students - 30,000 enrollment records in "takes" - 200 courses - 100 sections - 50 instructors - 20 departments

Write a SQL query to: Find all courses where every section ever offered had at least one student who received an 'A' grade. Please use standard PostgreSQL syntax.

AI-Generated SQL:

```
SELECT c.course_id, c.title
FROM course AS c
WHERE EXISTS (
    SELECT 1
    FROM section AS s
    WHERE s.course_id = c.course_id
)
AND NOT EXISTS (
    SELECT 1
    FROM section AS s
    WHERE s.course_id = c.course_id
      AND NOT EXISTS (
          SELECT 1
          FROM takes AS t
          WHERE t.course_id = s.course_id
            AND t.sec_id    = s.sec_id
            AND t.semester  = s.semester
            AND t.year      = s.year
            AND t.grade     = 'A'
      )
);
```

Note:

Used ChatGPT 5.3 April 1, 2026

Query 4 — Window Function

Prompt:

Given the following PostgreSQL database schema: create table classroom 2 (building varchar(15), 3 room_number varchar(7), 4 capacity numeric(4,0), 5 primary key (building, room_number) 6); 7 8 create table department 9 (dept_name varchar(20), 10 building varchar(15), 11 budget numeric(12,2) check (budget > 0), 12 primary key (dept_name) 13); 14 15 create table course 16 (course_id varchar(8), 17 title varchar(50), 18 dept_name varchar(20), 19 credits numeric(2,0) check (credits > 0), 20 primary key (course_id), 21 foreign key (dept_name) references department (dept_name) 22 on delete set null 23); 24 25 create table instructor 26 (ID varchar(5), 27 name varchar(20) not null, 28 dept_name varchar(20), 29 salary numeric(8,2) check (salary > 29000), 30 primary key (ID), 31 foreign key (dept_name) references department (dept_name) 32 on delete set null 33); 34 35 create table section 36 (course_id varchar(8), 37 sec_id varchar(8), 38 semester varchar(6) 39 check (semester in ('Fall', 'Winter', 'Spring', 'Summer')), 40 year numeric(4,0) check (year > 1701 and year < 2100), 41 building varchar(15), 42 room_number varchar(7), 43 time_slot_id varchar(4), 44 primary key (course_id, sec_id, semester, year), 45 foreign key (course_id) references course (course_id) 46 on delete cascade, 47 foreign key (building, room_number) 48 references classroom (building, room_number) 49 on delete set null 50); 51 52 create table teaches 53 (ID varchar(5), 54 course_id varchar(8), 55 sec_id varchar(8), 56 semester varchar(6), 57 year numeric(4,0), 58 primary key (ID, course_id, sec_id, semester, year), 59 foreign key (course_id, sec_id, semester, year) 60 references section (course_id, sec_id, semester, year) 61 on delete cascade, 62 foreign key (ID) references instructor (ID) 63 on delete cascade 64); 65 66 create table student 67 (ID varchar(5), 68 name varchar(20) not null, 69 dept_name varchar(20), 70 tot_cred numeric(3,0) check (tot_cred >= 0), 71 primary key (ID), 72 foreign key (dept_name) references department (dept_name) 73 on delete set null 74); 75 76 create table takes 77 (ID varchar(5), 78 course_id varchar(8), 79 sec_id varchar(8), 80 semester varchar(6), 81 year numeric(4,0), 82 grade varchar(2), 83 primary key (ID, course_id, sec_id, semester, year), 84 foreign key (course_id, sec_id, semester, year) 85 references section (course_id, sec_id, semester, year) 86 on delete cascade, 87 foreign key (ID) references student (ID) 88 on delete cascade 89); 90 91 create table advisor 92 (s_ID varchar(5), 93 i_ID varchar(5), 94 primary key (s_ID), 95 foreign key (i_ID) references instructor (ID) 96 on delete set null, 97 foreign key (s_ID) references student (ID) 98 on delete cascade 99); 100 101 create table time_slot 102 (time_slot_id varchar(4), 103 day varchar(1), 104 start_hr numeric(2) check (start_hr >= 0 and start_hr < 24), 105 start_min numeric(2) check (start_min >= 0 and start_min < 60), 106 end_hr numeric(2) check (end_hr >= 0 and end_hr < 24), 107 end_min numeric(2) check (end_min >= 0 and end_min < 60), 108 primary key (time_slot_id, day, start_hr, start_min) 109); 110 111 create table prereq 112 (course_id varchar(8), 113 prereq_id varchar(8), 114 primary key (course_id, prereq_id), 115 foreign key (course_id) references course (course_id) 116 on delete cascade, 117 foreign key (prereq_id) references course (course_id) 118);

The database contains approximately: - 2,000 students - 30,000 enrollment records in "takes" - 200 courses - 100 sections - 50 instructors - 20 departments

Write a SQL query to: "For each department, rank instructors by salary from highest to lowest. Show the instructor's name, department, salary, their rank within the department, and what percentile they fall into." Please use standard PostgreSQL syntax.

AI-Generated SQL:

```
SELECT
    name,
    dept_name AS department,
    salary,
    RANK() OVER (
        PARTITION BY dept_name
        ORDER BY salary DESC
    ) AS salary_rank,
    ROUND(
        (100 * PERCENT_RANK() OVER (
            PARTITION BY dept_name
            ORDER BY salary DESC
        ))::numeric,
        2
    ) AS percentile_in_department
FROM instructor
WHERE dept_name IS NOT NULL
ORDER BY dept_name, salary_rank, name;
```

Note:

Used ChatGPT 5.3 April 1, 2026

Query 5 — Complex Query

Prompt:

Given the following PostgreSQL database schema: create table classroom 2 (building varchar(15), 3 room_number varchar(7), 4 capacity numeric(4,0), 5 primary key (building, room_number) 6); 7 8 create table department 9 (dept_name varchar(20), 10 building varchar(15), 11 budget numeric(12,2) check (budget $>$ 0), 12 primary key (dept_name) 13); 14 15 create table course 16 (course_id varchar(8), 17 title varchar(50), 18 dept_name varchar(20), 19 credits numeric(2,0) check (credits $>$ 0), 20 primary key (course_id), 21 foreign key (dept_name) references department (dept_name) 22 on delete set null 23); 24 25 create table instructor 26 (ID varchar(5), 27 name varchar(20) not null, 28 dept_name varchar(20), 29 salary numeric(8,2) check (salary $>$ 29000), 30 primary key (ID), 31 foreign key (dept_name) references department (dept_name) 32 on delete set null 33); 34 35 create table section 36 (course_id varchar(8), 37 sec_id varchar(8), 38 semester varchar(6) 39 check (semester in ('Fall', 'Winter', 'Spring', 'Summer')), 40 year numeric(4,0) check (year $>$ 1701 and year $<$ 2100), 41 building varchar(15), 42 room_number varchar(7), 43 time_slot_id varchar(4), 44 primary key (course_id, sec_id,

semester, year), 45 foreign key (course_id) references course (course_id) 46 on delete cascade, 47 foreign key (building, room_number) 48 references classroom (building, room_number) 49 on delete set null 50); 51 52 create table teaches 53 (ID varchar(5), 54 course_id varchar(8), 55 sec_id varchar(8), 56 semester varchar(6), 57 year numeric(4,0), 58 primary key (ID, course_id, sec_id, semester, year), 59 foreign key (course_id, sec_id, semester, year) 60 references section (course_id, sec_id, semester, year) 61 on delete cascade, 62 foreign key (ID) references instructor (ID) 63 on delete cascade 64); 65 66 create table student 67 (ID varchar(5), 68 name varchar(20) not null, 69 dept_name varchar(20), 70 tot_cred numeric(3,0) check (tot_cred >= 0), 71 primary key (ID), 72 foreign key (dept_name) references department (dept_name) 73 on delete set null 74); 75 76 create table takes 77 (ID varchar(5), 78 course_id varchar(8), 79 sec_id varchar(8), 80 semester varchar(6), 81 year numeric(4,0), 82 grade varchar(2), 83 primary key (ID, course_id, sec_id, semester, year), 84 foreign key (course_id, sec_id, semester, year) 85 references section (course_id, sec_id, semester, year) 86 on delete cascade, 87 foreign key (ID) references student (ID) 88 on delete cascade 89); 90 91 create table advisor 92 (s_ID varchar(5), 93 i_ID varchar(5), 94 primary key (s_ID), 95 foreign key (i_ID) references instructor (ID) 96 on delete set null, 97 foreign key (s_ID) references student (ID) 98 on delete cascade 99); 100 101 create table time_slot 102 (time_slot_id varchar(4), 103 day varchar(1), 104 start_hr numeric(2) check (start_hr >= 0 and start_hr < 24), 105 start_min numeric(2) check (start_min >= 0 and start_min < 60), 106 end_hr numeric(2) check (end_hr >= 0 and end_hr < 24), 107 end_min numeric(2) check (end_min >= 0 and end_min < 60), 108 primary key (time_slot_id, day, start_hr, start_min) 109); 110 111 create table prereq 112 (course_id varchar(8), 113 prereq_id varchar(8), 114 primary key (course_id, prereq_id), 115 foreign key (course_id) references course (course_id) 116 on delete cascade, 117 foreign key (prereq_id) references course (course_id) 118);

The database contains approximately: - 2,000 students - 30,000 enrollment records in "takes" - 200 courses - 100 sections - 50 instructors - 20 departments

Write a SQL query to: "Find students who have taken courses offered by at least 3 different departments but have never taken a course taught by their own advisor." Please use standard PostgreSQL syntax.

AI-Generated SQL:

```
SELECT s.ID, s.name
FROM student AS s
JOIN takes AS t
  ON t.ID = s.ID
JOIN course AS c
  ON c.course_id = t.course_id
GROUP BY s.ID, s.name
HAVING COUNT(DISTINCT c.dept_name) >= 3
   AND NOT EXISTS (
```

```
    SELECT 1
    FROM advisor AS a
    JOIN teaches AS te
      ON te.ID = a.i_ID
    JOIN takes AS t2
      ON t2.ID = a.s_ID
     AND t2.course_id = te.course_id
     AND t2.sec_id = te.sec_id
     AND t2.semester = te.semester
     AND t2.year = te.year
    WHERE a.s_ID = s.ID
);
```

Note:

Used ChatGPT 5.3 April 1, 2026

Part 2: Execution Plan Analysis

Query 1 — Aggregation

Execution Plan:

	QUERY PLAN text
1	Sort (cost=1119.60..1119.65 rows=20 width=49) (actual time=34.919..34.924 rows=20.00 loops=1)
2	Sort Key: (count(*)) DESC
3	Sort Method: quicksort Memory: 26kB
4	Buffers: shared hit=473
5	-> HashAggregate (cost=1118.92..1119.17 rows=20 width=49) (actual time=34.410..34.425 rows=20.00 loops=1)
6	Group Key: s.dept_name
7	Batches: 1 Memory Usage: 32kB
8	Buffers: shared hit=470
9	-> Hash Join (cost=75.00..893.92 rows=30000 width=13) (actual time=3.985..23.827 rows=30000.00 loops=1)
10	Hash Cond: ((t.id)::text = (s.id)::text)
11	Buffers: shared hit=470
12	-> Seq Scan on takes t (cost=0.00..740.00 rows=30000 width=5) (actual time=2.560..9.954 rows=30000.00 loops=…
13	Buffers: shared hit=440
14	-> Hash (cost=50.00..50.00 rows=2000 width=18) (actual time=1.380..1.381 rows=2000.00 loops=1)
15	Buckets: 2048 Batches: 1 Memory Usage: 119kB
16	Buffers: shared hit=30
17	-> Seq Scan on student s (cost=0.00..50.00 rows=2000 width=18) (actual time=0.171..0.754 rows=2000.00 loo…
18	Buffers: shared hit=30
19	Planning:
20	Buffers: shared hit=198 dirtied=1
21	Planning Time: 7.732 ms
22	Execution Time: 35.395 ms

Performance/Correctness Issue:

1. Joining student to takes: it changes the unit of analysis from averaging over students to averaging over enrollments, which is not what the prompt is asking
2. Sequential scan on takes: it unnecessarily scanned all 30,000 rows when it could have used a filter

Query 2 — Multi-Table Join

Execution Plan:

	QUERY PLAN text
1	Sort (cost=3017.33..3042.33 rows=10000 width=63) (actual time=47.262..47.279 rows=100.00 loops=1)
2	Sort Key: i.name, s.year, s.semester, c.course_id, s.sec_id
3	Sort Method: quicksort Memory: 33kB
4	Buffers: shared hit=455
5	-> HashAggregate (cost=1977.94..2352.94 rows=10000 width=63) (actual time=46.483..46.701 rows=100.00 loops=1)
6	Group Key: s.year, s.semester, c.course_id, s.sec_id, i.id
7	Filter: (count(tk.id) >= 10)
8	Batches: 1 Memory Usage: 1065kB
9	Buffers: shared hit=449
10	-> Hash Join (cost=22.22..1527.94 rows=30000 width=60) (actual time=1.342..25.539 rows=30000.00 loops=1)
11	Hash Cond: (((tk.course_id)::text = (te.course_id)::text) AND ((tk.sec_id)::text = (te.sec_id)::text) AND ((tk.semester)::text = (te.semester)::text) AND (tk.year = te.year))
12	Buffers: shared hit=449
13	-> Seq Scan on takes tk (cost=0.00..740.00 rows=30000 width=21) (actual time=0.104..3.270 rows=30000.00 loops=1)
14	Buffers: shared hit=440
15	-> Hash (cost=20.22..20.22 rows=100 width=75) (actual time=1.216..1.224 rows=100.00 loops=1)
16	Buckets: 1024 Batches: 1 Memory Usage: 19kB
17	Buffers: shared hit=9
18	-> Hash Join (cost=15.62..20.22 rows=100 width=75) (actual time=0.898..1.113 rows=100.00 loops=1)
19	Hash Cond: ((te.course_id)::text = (c.course_id)::text)
20	Buffers: shared hit=9
21	-> Hash Join (cost=7.12..11.46 rows=100 width=54) (actual time=0.695..0.863 rows=100.00 loops=1)
22	Hash Cond: (((te.course_id)::text = (s.course_id)::text) AND ((te.sec_id)::text = (s.sec_id)::text) AND ((te.semester)::text = (s.semester)::text) AND (te.year …
23	Buffers: shared hit=5
24	-> Hash Join (cost=2.12..5.41 rows=100 width=38) (actual time=0.387..0.481 rows=100.00 loops=1)
25	Hash Cond: ((te.id)::text = (i.id)::text)
26	Buffers: shared hit=3
27	-> Seq Scan on teaches te (cost=0.00..3.00 rows=100 width=21) (actual time=0.019..0.042 rows=100.00 loops=1)
28	Buffers: shared hit=2
29	-> Hash (cost=1.50..1.50 rows=50 width=22) (actual time=0.080..0.080 rows=50.00 loops=1)
30	Buckets: 1024 Batches: 1 Memory Usage: 11kB
31	Buffers: shared hit=1
32	-> Seq Scan on instructor i (cost=0.00..1.50 rows=50 width=22) (actual time=0.038..0.047 rows=50.00 loops=1)
33	Buffers: shared hit=1
34	-> Hash (cost=3.00..3.00 rows=100 width=16) (actual time=0.088..0.092 rows=100.00 loops=1)
35	Buckets: 1024 Batches: 1 Memory Usage: 13kB
36	Buffers: shared hit=2
37	-> Seq Scan on section s (cost=0.00..3.00 rows=100 width=16) (actual time=0.030..0.055 rows=100.00 loops=1)
38	Buffers: shared hit=2
39	-> Hash (cost=6.00..6.00 rows=200 width=21) (actual time=0.185..0.185 rows=200.00 loops=1)
40	Buckets: 1024 Batches: 1 Memory Usage: 19kB
41	Buffers: shared hit=4
42	-> Seq Scan on course c (cost=0.00..6.00 rows=200 width=21) (actual time=0.037..0.130 rows=200.00 loops=1)
43	Buffers: shared hit=4
44	Planning:
45	Buffers: shared hit=235 dirtied=6
46	Planning Time: 9.178 ms
47	Execution Time: 48.096 ms

Performance/Correctness Issue:

1. Sequential scan on takes: it reads all 30,000 rows when it's unnecessary.
2. Sort Key: unnecessary order by at the end, can be replaced by Index

Query 3 — Correlated Subquery

Execution Plan:

	QUERY PLAN text
1	Hash Semi Join (cost=12.75..834.70 rows=49 width=21) (actual time=12.167..12.175 rows=0.00 loops=1)
2	Hash Cond: ((c.course_id)::text = (s.course_id)::text)
3	Buffers: shared hit=448
4	-> Hash Right Anti Join (cost=8.50..829.48 rows=115 width=21) (actual time=12.023..12.044 rows=115.00 loops=1)
5	Hash Cond: ((s_1.course_id)::text = (c.course_id)::text)
6	Buffers: shared hit=446
7	-> Nested Loop Anti Join (cost=0.00..820.25 rows=100 width=4) (actual time=11.044..11.087 rows=100.00 loops=1)
8	Join Filter: (((t.course_id)::text = (s_1.course_id)::text) AND ((t.sec_id)::text = (s_1.sec_id)::text) AND ((t.semester)::text = (s_1.semester)::text) AND (t.year = s_...
9	Buffers: shared hit=442
10	-> Seq Scan on section s_1 (cost=0.00..3.00 rows=100 width=16) (actual time=0.008..0.018 rows=100.00 loops=1)
11	Buffers: shared hit=2
12	-> Materialize (cost=0.00..815.00 rows=1 width=16) (actual time=0.110..0.110 rows=0.00 loops=100)
13	Storage: Memory Maximum Storage: 17kB
14	Buffers: shared hit=440
15	-> Seq Scan on takes t (cost=0.00..815.00 rows=1 width=16) (actual time=11.028..11.028 rows=0.00 loops=1)
16	Filter: ((grade)::text = 'A'::text)
17	Rows Removed by Filter: 30000
18	Buffers: shared hit=440
19	-> Hash (cost=6.00..6.00 rows=200 width=21) (actual time=0.902..0.903 rows=200.00 loops=1)
20	Buckets: 1024 Batches: 1 Memory Usage: 19kB
21	Buffers: shared hit=4
22	-> Seq Scan on course c (cost=0.00..6.00 rows=200 width=21) (actual time=0.033..0.854 rows=200.00 loops=1)
23	Buffers: shared hit=4
24	-> Hash (cost=3.00..3.00 rows=100 width=4) (actual time=0.102..0.107 rows=100.00 loops=1)
25	Buckets: 1024 Batches: 1 Memory Usage: 12kB
26	Buffers: shared hit=2
27	-> Seq Scan on section s (cost=0.00..3.00 rows=100 width=4) (actual time=0.034..0.055 rows=100.00 loops=1)
28	Buffers: shared hit=2
29	Planning:
30	Buffers: shared hit=371 dirtied=1
31	Planning Time: 5.109 ms
32	Execution Time: 12.257 ms

Total rows: 32 | Query complete 00:00:00.074

Performance/Correctness Issue:

1. Grade comparison: The AI SQL is filtering by 'A' but it's not including 'A+' or 'A-", so it's returning 0 rows. This happens when AI doesn't know the data format.
2. Correlated subqueries: The plan included anti join logic (NOT EXIST, EXIST). Using HAVING instead is faster

Query 4 — Window Function

Execution Plan:

	QUERY PLAN text
1	Incremental Sort (cost=3.04..5.77 rows=50 width=66) (actual time=0.201..0.230 rows=50.00 loops=1)
2	Sort Key: dept_name, (rank() OVER w1), name
3	Presorted Key: dept_name
4	Full-sort Groups: 2 Sort Method: quicksort Average Memory: 27kB Peak Memory: 27kB
5	Buffers: shared hit=1
6	-> WindowAgg (cost=2.93..4.41 rows=50 width=66) (actual time=0.087..0.161 rows=50.00 loops=1)
7	Window: w1 AS (PARTITION BY dept_name ORDER BY salary ROWS UNBOUNDED PRECEDING)
8	Storage: Memory Maximum Storage: 17kB
9	Buffers: shared hit=1
10	-> Sort (cost=2.91..3.04 rows=50 width=26) (actual time=0.070..0.075 rows=50.00 loops=1)
11	Sort Key: dept_name, salary DESC
12	Sort Method: quicksort Memory: 27kB
13	Buffers: shared hit=1
14	-> Seq Scan on instructor (cost=0.00..1.50 rows=50 width=26) (actual time=0.015..0.023 rows=50.00 loo…
15	Filter: (dept_name IS NOT NULL)
16	Buffers: shared hit=1
17	Planning Time: 0.084 ms
18	Execution Time: 0.261 ms

Total rows: 18 | Query complete 00:00:00.098

Performance/Correctness Issue:

1. Percentile logic: The query is using PERCENT_RANK(), meaning that the highest paid instructor is given the rank of 0 and the lowest will get a rank of 1. This is backwards from what the prompt was asking, which was be percentile, not rank

2. Department NULL filter: The query includes the filter (dept_name IS NOT NULL). It's unnecessary if the column is already non null by schema and could exclude rows if null departments were meant to be shown

Query 5 — Complex Query

Execution Plan:

	QUERY PLAN text
1	GroupAggregate (cost=3213.74..36138.24 rows=333 width=11) (actual time=89.316..110.135 rows=1580.00 loops=1)
2	Group Key: s.id
3	Filter: ((count(DISTINCT c.dept_name) >= 3) AND (NOT (ANY ((s.id)::text = (hashed SubPlan 2).col1))))
4	Rows Removed by Filter: 420
5	Buffers: shared hit=938
6	-> Sort (cost=3213.74..3288.74 rows=30000 width=20) (actual time=54.484..61.390 rows=30000.00 loops=1)
7	Sort Key: s.id, c.dept_name
8	Sort Method: quicksort Memory: 1987kB
9	Buffers: shared hit=474
10	-> Hash Join (cost=83.50..982.84 rows=30000 width=20) (actual time=0.903..28.131 rows=30000.00 loops=1)
11	Hash Cond: ((t.course_id)::text = (c.course_id)::text)
12	Buffers: shared hit=474
13	-> Hash Join (cost=75.00..893.92 rows=30000 width=15) (actual time=0.754..17.847 rows=30000.00 loops=1)
14	Hash Cond: ((t.id)::text = (s.id)::text)
15	Buffers: shared hit=470
16	-> Seq Scan on takes t (cost=0.00..740.00 rows=30000 width=9) (actual time=0.104..3.619 rows=30000.00 loops=1)
17	Buffers: shared hit=440
18	-> Hash (cost=50.00..50.00 rows=2000 width=11) (actual time=0.644..0.645 rows=2000.00 loops=1)
19	Buckets: 2048 Batches: 1 Memory Usage: 104kB
20	Buffers: shared hit=30
21	-> Seq Scan on student s (cost=0.00..50.00 rows=2000 width=11) (actual time=0.011..0.264 rows=2000.00 loops=1)
22	Buffers: shared hit=30
23	-> Hash (cost=6.00..6.00 rows=200 width=13) (actual time=0.142..0.143 rows=200.00 loops=1)
24	Buckets: 1024 Batches: 1 Memory Usage: 17kB
25	Buffers: shared hit=4
26	-> Seq Scan on course c (cost=0.00..6.00 rows=200 width=13) (actual time=0.017..0.057 rows=200.00 loops=1)
27	Buffers: shared hit=4
28	SubPlan 2
29	-> Hash Join (cost=72.25..1284.92 rows=31 width=32) (actual time=0.747..34.266 rows=613.00 loops=1)
30	Hash Cond: (((a.i_id)::text = (te.id)::text) AND ((t2.course_id)::text = (te.course_id)::text) AND ((t2.sec_id)::text = (te.sec_id)::text) AND ((t2.semester)::text = (te.semester)::te…
31	Buffers: shared hit=464
32	-> Hash Join (cost=67.00..885.92 rows=30000 width=26) (actual time=0.537..21.443 rows=30000.00 loops=1)

33	Hash Cond: ((t2.id)::text = (a.s_id)::text)
34	Buffers: shared hit=462
35	-> Seq Scan on takes t2 (cost=0.00..740.00 rows=30000 width=21) (actual time=0.094..4.244 rows=30000.00 loops=1)
36	Buffers: shared hit=440
37	-> Hash (cost=42.00..42.00 rows=2000 width=10) (actual time=0.437..0.438 rows=2000.00 loops=1)
38	Buckets: 2048 Batches: 1 Memory Usage: 102kB
39	Buffers: shared hit=22
40	-> Seq Scan on advisor a (cost=0.00..42.00 rows=2000 width=10) (actual time=0.011..0.169 rows=2000.00 loops=1)
41	Buffers: shared hit=22
42	-> Hash (cost=3.00..3.00 rows=100 width=21) (actual time=0.061..0.061 rows=100.00 loops=1)
43	Buckets: 1024 Batches: 1 Memory Usage: 14kB
44	Buffers: shared hit=2
45	-> Seq Scan on teaches te (cost=0.00..3.00 rows=100 width=21) (actual time=0.016..0.025 rows=100.00 loops=1)
46	Buffers: shared hit=2
47	Planning:
48	Buffers: shared hit=36
49	Planning Time: 1.604 ms
50	Execution Time: 110.588 ms
Total rows: 50	Query complete 00:00:00.248

Performance/Correctness Issue:

1. Multiple sequential scan on takes: The query scans all 30,000 rows twice for 'takes'. This can be consolidated by joining
2. Not exist: increases complexity and lads to multiple re scanning of 'takes'.

Part 3: Query Optimization

Query 1 — Aggregation

Indexes created

CREATE INDEX idx_takes_student_id ON takes(ID);

CREATE INDEX idx_course_dept ON course(dept_name);

Optimized Version

New Execution Plan

Performance Comparison

Metric	Before	After	Change
Scan type on takes	seq scan	index scan	
Join students to take	avg(s.tot_cred)	use CTE	
Rows scanned	30000	30000	0%
Execution Tim	274	92	-66%

Query 2 — Multi-Table Join

Indexes created

CREATE INDEX idx_takes_student_id ON takes(ID)

CREATE INDEX idx_takes_section

ON takes(course_id, sec_id, semester, year);

Optimized Version

```
SELECT
    i.name AS instructor_name,
    i.dept_name AS instructor_department,
    c.course_id,
    c.title AS course_title,
    s.sec_id,
    s.semester,
    s.year,
    COUNT(tk.ID) AS enrolled_students
FROM instructor AS i
JOIN teaches AS te
    ON i.ID = te.ID
JOIN section AS s
    ON te.course_id = s.course_id
   AND te.sec_id = s.sec_id
   AND te.semester = s.semester
   AND te.year = s.year
JOIN course AS c
    ON s.course_id = c.course_id
JOIN takes AS tk
    ON s.course_id = tk.course_id
   AND s.sec_id = tk.sec_id
   AND s.semester = tk.semester
```

```
   AND s.year = tk.year
GROUP BY
    i.ID,
    i.name,
    i.dept_name,
    c.course_id,
    c.title,
    s.sec_id,
    s.semester,
    s.year
HAVING COUNT(tk.ID) >= 10
ORDER BY i.dept_name DESC, i.name;
```

New Execution Plan

	QUERY PLAN text
1	Sort (cost=3019.98..3044.98 rows=10000 width=63) (actual time=58.591..58.602 rows=100.00 loops=1)
2	Sort Key: i.dept_name DESC, i.name
3	Sort Method: quicksort Memory: 33kB
4	Buffers: shared hit=16790
5	-> HashAggregate (cost=1980.59..2355.59 rows=10000 width=63) (actual time=58.340..58.523 rows=100.00 loops=1)
6	Group Key: i.id, c.course_id, s.sec_id, s.semester, s.year
7	Filter: (count(tk.id) >= 10)
8	Batches: 1 Memory Usage: 1065kB
9	Buffers: shared hit=16790
10	-> Nested Loop (cost=15.92..1530.59 rows=30000 width=60) (actual time=0.255..40.749 rows=30000.00 loops=1)
11	Buffers: shared hit=16790
12	-> Hash Join (cost=15.62..20.22 rows=100 width=75) (actual time=0.198..0.611 rows=100.00 loops=1)
13	Hash Cond: ((te.course_id)::text = (c.course_id)::text)
14	Buffers: shared hit=9
15	-> Hash Join (cost=7.12..11.46 rows=100 width=54) (actual time=0.104..0.431 rows=100.00 loops=1)
16	Hash Cond: (((te.course_id)::text = (s.course_id)::text) AND ((te.sec_id)::text = (s.sec_id)::text) AND ((te.semester)::text = (s.semester)::text) AND (te.year = …
17	Buffers: shared hit=5
18	-> Hash Join (cost=2.12..5.41 rows=100 width=38) (actual time=0.041..0.192 rows=100.00 loops=1)
19	Hash Cond: ((te.id)::text = (i.id)::text)
20	Buffers: shared hit=3
21	-> Seq Scan on teaches te (cost=0.00..3.00 rows=100 width=21) (actual time=0.005..0.057 rows=100.00 loops=1)
22	Buffers: shared hit=2
23	-> Hash (cost=1.50..1.50 rows=50 width=22) (actual time=0.026..0.027 rows=50.00 loops=1)
24	Buckets: 1024 Batches: 1 Memory Usage: 11kB
25	Buffers: shared hit=1
26	-> Seq Scan on instructor i (cost=0.00..1.50 rows=50 width=22) (actual time=0.008..0.014 rows=50.00 loops=1)
27	Buffers: shared hit=1
28	-> Hash (cost=3.00..3.00 rows=100 width=16) (actual time=0.052..0.052 rows=100.00 loops=1)
29	Buckets: 1024 Batches: 1 Memory Usage: 13kB
30	Buffers: shared hit=2
31	-> Seq Scan on section s (cost=0.00..3.00 rows=100 width=16) (actual time=0.005..0.020 rows=100.00 loops=1)
32	Buffers: shared hit=2

```
33                  ->  Hash  (cost=6.00..6.00 rows=200 width=21) (actual time=0.085..0.085 rows=200.00 loops=1)
34                        Buckets: 1024  Batches: 1  Memory Usage: 19kB
35                        Buffers: shared hit=4
36                        ->  Seq Scan on course c  (cost=0.00..6.00 rows=200 width=21) (actual time=0.017..0.043 rows=200.00 loops=1)
37                              Buffers: shared hit=4
38              ->  Memoize  (cost=0.30..17.65 rows=6 width=21) (actual time=0.032..0.345 rows=300.00 loops=100)
39                    Cache Key: te.course_id, te.sec_id, te.semester, te.year
40                    Cache Mode: logical
41                    Hits: 0  Misses: 100  Evictions: 0  Overflows: 0  Memory Usage: 1616kB
42                    Buffers: shared hit=16781
43                    ->  Index Scan using idx_takes_section on takes tk  (cost=0.29..17.64 rows=6 width=21) (actual time=0.030..0.210 rows=300.00 loops=100)
44                          Index Cond: (((course_id)::text = (te.course_id)::text) AND ((sec_id)::text = (te.sec_id)::text) AND ((semester)::text = (te.semester)::text) AND (year = te.year))
45                          Index Searches: 100
46                          Buffers: shared hit=16781
47  Planning:
48    Buffers: shared hit=32
49  Planning Time: 3.140 ms
50  Execution Time: 59.155 ms
```

Performance Comparison

Metric	Before	After	Change
Scan type on takes	Seq scan	Index Scan	
Rows scanned	30,000	300	-99%
Execution Time	173ms	59ms	-65.9

Query 3 — Correlated Subquery

Indexes created

CREATE INDEX idx_takes_grade_covering

ON takes(course_id, sec_id, semester, year)

INCLUDE (ID, grade);

Optimized Version

```
SELECT c.course_id, c.title
FROM course c
JOIN section sec ON c.course_id = sec.course_id
LEFT JOIN takes t ON sec.course_id = t.course_id
AND sec.sec_id = t.sec_id
AND sec.semester = t.semester
```

```
AND sec.year = t.year
AND t.grade LIKE 'A%'
GROUP BY c.course_id, c.title
HAVING COUNT(DISTINCT sec.course_id || sec.sec_id || sec.semester || sec.year)
= COUNT(DISTINCT CASE WHEN t.grade LIKE 'A%'
THEN sec.course_id || sec.sec_id || sec.semester || sec.year
END);
```

New Execution Plan

	QUERY PLAN text
1	GroupAggregate (cost=1616.44..1990.34 rows=1 width=21) (actual time=20.259..24.998 rows=85.00 loops=1)
2	Group Key: c.course_id
3	Filter: (count(DISTINCT (((((sec.course_id)::text || (sec.sec_id)::text) || (sec.semester)::text) || (sec.year)::text))) = count(DISTINCT CASE WHEN ((t.grade)::text ~~ 'A%'::text) THEN (((((s…
4	Buffers: shared hit=446
5	-> Sort (cost=1616.44..1641.20 rows=9904 width=40) (actual time=20.130..20.983 rows=9904.00 loops=1)
6	Sort Key: c.course_id, (((((sec.course_id)::text || (sec.sec_id)::text) || (sec.semester)::text) || (sec.year)::text))
7	Sort Method: quicksort Memory: 1094kB
8	Buffers: shared hit=446
9	-> Hash Join (cost=13.50..959.12 rows=9904 width=40) (actual time=0.233..14.952 rows=9904.00 loops=1)
10	Hash Cond: ((sec.course_id)::text = (c.course_id)::text)
11	Buffers: shared hit=446
12	-> Hash Right Join (cost=5.00..924.06 rows=9904 width=19) (actual time=0.158..10.030 rows=9904.00 loops=1)
13	Hash Cond: (((t.course_id)::text = (sec.course_id)::text) AND ((t.sec_id)::text = (sec.sec_id)::text) AND ((t.semester)::text = (sec.semester)::text) AND (t.year = sec.year))
14	Buffers: shared hit=442
15	-> Seq Scan on takes t (cost=0.00..815.00 rows=9904 width=19) (actual time=0.094..4.545 rows=9904.00 loops=1)
16	Filter: ((grade)::text ~~ 'A%'::text)
17	Rows Removed by Filter: 20096
18	Buffers: shared hit=440
19	-> Hash (cost=3.00..3.00 rows=100 width=16) (actual time=0.058..0.058 rows=100.00 loops=1)
20	Buckets: 1024 Batches: 1 Memory Usage: 13kB
21	Buffers: shared hit=2
22	-> Seq Scan on section sec (cost=0.00..3.00 rows=100 width=16) (actual time=0.012..0.026 rows=100.00 loops=1)
23	Buffers: shared hit=2
24	-> Hash (cost=6.00..6.00 rows=200 width=21) (actual time=0.070..0.071 rows=200.00 loops=1)
25	Buckets: 1024 Batches: 1 Memory Usage: 19kB
26	Buffers: shared hit=4
27	-> Seq Scan on course c (cost=0.00..6.00 rows=200 width=21) (actual time=0.006..0.031 rows=200.00 loops=1)
28	Buffers: shared hit=4
29	Planning:
30	Buffers: shared hit=12
31	Planning Time: 0.487 ms
32	Execution Time: 25.198 ms

Performance Comparison

Metric	Before	After	Change
Grade comparison	Returned 0 rows	Returned 85 rowss	85%
Correlated Subquris	NOT EXIST, EXIST	HAVING	

Metric	Before	After	Change
Execution Tim	173ms	25ms	85%

Query 4 — Window Function

Indexes created

CREATE INDEX idx_instructor_dept_salary

ON instructor(dept_name, salary DESC);

Optimized Version

```
SELECT
    name,
    dept_name AS department,
    salary,
    RANK() OVER (
        PARTITION BY dept_name
        ORDER BY salary DESC
    ) AS salary_rank,
    ROUND(
        (100 * CUME_DIST() OVER (
            PARTITION BY dept_name
            ORDER BY salary
        ))::numeric,
        2
    ) AS percentile_in_department
FROM instructor
WHERE dept_name IS NOT NULL
ORDER BY dept_name, salary_rank, name;
```

New Execution Plan

	QUERY PLAN text
1	Incremental Sort (cost=3.30..7.87 rows=50 width=66) (actual time=0.344..0.465 rows=50.00 loops=1)
2	Sort Key: dept_name, (rank() OVER w1), name
3	Presorted Key: dept_name
4	Full-sort Groups: 2 Sort Method: quicksort Average Memory: 27kB Peak Memory: 27kB
5	Buffers: shared hit=1
6	-> WindowAgg (cost=3.07..6.52 rows=50 width=66) (actual time=0.200..0.406 rows=50.00 loops=1)
7	Window: w2 AS (PARTITION BY dept_name ORDER BY salary ROWS UNBOUNDED PRECEDING)
8	Storage: Memory Maximum Storage: 17kB
9	Buffers: shared hit=1
10	-> Incremental Sort (cost=3.01..5.27 rows=50 width=34) (actual time=0.189..0.289 rows=50.00 loops=1)
11	Sort Key: dept_name, salary
12	Presorted Key: dept_name
13	Full-sort Groups: 2 Sort Method: quicksort Average Memory: 26kB Peak Memory: 26kB
14	Buffers: shared hit=1
15	-> WindowAgg (cost=2.93..3.91 rows=50 width=34) (actual time=0.053..0.098 rows=50.00 loops=1)
16	Window: w1 AS (PARTITION BY dept_name ORDER BY salary ROWS UNBOUNDED PRECEDING)
17	Storage: Memory Maximum Storage: 17kB
18	Buffers: shared hit=1
19	-> Sort (cost=2.91..3.04 rows=50 width=26) (actual time=0.049..0.053 rows=50.00 loops=1)
20	Sort Key: dept_name, salary DESC
21	Sort Method: quicksort Memory: 27kB
22	Buffers: shared hit=1
23	-> Seq Scan on instructor (cost=0.00..1.50 rows=50 width=26) (actual time=0.014..0.022 rows=50.00 loo…
24	Filter: (dept_name IS NOT NULL)
25	Buffers: shared hit=1

Performance Comparison

Metric	Before	After	Change
Percentile logic	0= highest 1= lowest	0= lowest 1=highest	
Rows scanned	50	50	0
Execution Tim	261	180	31%

Query 5 — Complex Query

Indexes created

CREATE INDEX idx_advisor_instructor ON advisor(i_ID);

Optimized Version

```
WITH dept_counts AS (
    SELECT s.ID,s.name,COUNT(DISTINCT c.dept_name) AS dept_count
    FROM student AS s
    JOIN takes AS t ON t.ID = s.ID
    JOIN course AS c ON c.course_id = t.course_id
    GROUP BY s.ID, s.name
),
advisor_taught_students AS (
    SELECT DISTINCT a.s_ID AS student_id
    FROM advisor AS a
    JOIN teaches AS te ON te.ID = a.i_ID
    JOIN takes AS t ON t.ID = a.s_ID
       AND t.course_id = te.course_id
       AND t.sec_id = te.sec_id
       AND t.semester = te.semester
       AND t.year = te.year
)
SELECT d.ID, d.name
FROM dept_counts AS d
LEFT JOIN advisor_taught_students AS ats ON ats.student_id = d.ID
WHERE d.dept_count >= 3
  AND ats.student_id IS NULL;
```

New Execution Plan

```
Merge Anti Join  (cost=4499.42..4759.02 rows=564 width=11) (actual time=69.753..82.378 rows=1580.00 loops=1)
  Merge Cond: ((s.id)::text = (a.s_id)::text)
  Buffers: shared hit=938
  ->  GroupAggregate  (cost=3213.74..3463.74 rows=667 width=19) (actual time=43.433..55.156 rows=2000.00 loops=1)
        Group Key: s.id
        Filter: (count(DISTINCT c.dept_name) >= 3)
        Buffers: shared hit=474
        ->  Sort  (cost=3213.74..3288.74 rows=30000 width=20) (actual time=43.417..47.056 rows=30000.00 loops=1)
              Sort Key: s.id, c.dept_name
              Sort Method: quicksort  Memory: 1987kB
              Buffers: shared hit=474
              ->  Hash Join  (cost=83.50..982.84 rows=30000 width=20) (actual time=0.983..22.595 rows=30000.00 loops=1)
                    Hash Cond: ((t.course_id)::text = (c.course_id)::text)
                    Buffers: shared hit=474
                    ->  Hash Join  (cost=75.00..893.92 rows=30000 width=15) (actual time=0.887..14.244 rows=30000.00 loops=1)
                          Hash Cond: ((t.id)::text = (s.id)::text)
                          Buffers: shared hit=470
                          ->  Seq Scan on takes t  (cost=0.00..740.00 rows=30000 width=9) (actual time=0.099..3.072 rows=30000.00 loops=1)
                                Buffers: shared hit=440
                          ->  Hash  (cost=50.00..50.00 rows=2000 width=11) (actual time=0.758..0.759 rows=2000.00 loops=1)
                                Buckets: 2048  Batches: 1  Memory Usage: 104kB
                                Buffers: shared hit=30
                                ->  Seq Scan on student s  (cost=0.00..50.00 rows=2000 width=11) (actual time=0.010..0.322 rows=2000.00 loops=1)
                                      Buffers: shared hit=30
                    ->  Hash  (cost=6.00..6.00 rows=200 width=13) (actual time=0.078..0.079 rows=200.00 loops=1)
                          Buckets: 1024  Batches: 1  Memory Usage: 17kB
                          Buffers: shared hit=4
                          ->  Seq Scan on course c  (cost=0.00..6.00 rows=200 width=13) (actual time=0.011..0.038 rows=200.00 loops=1)
                                Buffers: shared hit=4
  ->  Unique  (cost=1285.68..1285.84 rows=31 width=5) (actual time=26.316..26.657 rows=420.00 loops=1)
        Buffers: shared hit=464
        ->  Sort  (cost=1285.68..1285.76 rows=31 width=5) (actual time=26.314..26.380 rows=613.00 loops=1)
```

```
        Sort Key: a.s_id
        Sort Method: quicksort  Memory: 25kB
        Buffers: shared hit=464
        ->  Hash Join  (cost=72.25..1284.92 rows=31 width=5) (actual time=2.155..26.048 rows=613.00 loops=1)
              Hash Cond: (((a.i_id)::text = (te.id)::text) AND ((t_1.course_id)::text = (te.course_id)::text) AND ((t_1.sec_id)::text = (te.sec_id)::text) AND ((t_1.semester)::text = (te.s…
              Buffers: shared hit=464
              ->  Hash Join  (cost=67.00..885.92 rows=30000 width=26) (actual time=0.674..16.407 rows=30000.00 loops=1)
                    Hash Cond: ((t_1.id)::text = (a.s_id)::text)
                    Buffers: shared hit=462
                    ->  Seq Scan on takes t_1  (cost=0.00..740.00 rows=30000 width=21) (actual time=0.130..3.411 rows=30000.00 loops=1)
                          Buffers: shared hit=440
                    ->  Hash  (cost=42.00..42.00 rows=2000 width=10) (actual time=0.527..0.528 rows=2000.00 loops=1)
                          Buckets: 2048  Batches: 1  Memory Usage: 102kB
                          Buffers: shared hit=22
                          ->  Seq Scan on advisor a  (cost=0.00..42.00 rows=2000 width=10) (actual time=0.014..0.175 rows=2000.00 loops=1)
                                Buffers: shared hit=22
              ->  Hash  (cost=3.00..3.00 rows=100 width=21) (actual time=1.329..1.329 rows=100.00 loops=1)
                    Buckets: 1024  Batches: 1  Memory Usage: 14kB
                    Buffers: shared hit=2
                    ->  Seq Scan on teaches te  (cost=0.00..3.00 rows=100 width=21) (actual time=0.017..0.031 rows=100.00 loops=1)
                          Buffers: shared hit=2
Planning:
  Buffers: shared hit=32
Planning Time: 1.180 ms
Execution Time: 82.636 ms
```

Performance Comparison

Metric	Before	After	Change
Scan type on advisor	seq scan	index scan	
Rows scanned	2000	2000	0
Execution Tim	248ms	105ms	-58%

Part 4: Reflection

Across all five queries, the AI consistently produced syntactically correct and logically reasonable SQL, especially for joins, grouping, and window functions. It often selected appropriate strategies, such as using `GROUP BY`, `HAVING`, window functions (`RANK`, `PERCENT_RANK`), and anti-join patterns (`NOT EXISTS`) for "for all" conditions. However, AI consistently missed deeper correctness and performance details. The most common correctness issue was mixing levels of aggregation, such as computing `AVG(tot_cred)` after joining to `takes`, which weighted students by their number of enrollments. It also misused functions like `PERCENT_RANK()` which produced values that did not match the intended meaning of "percentile." On the performance side, AI repeatedly generated queries that caused full table scans on `takes` and overly complex

sub queries, such as `NOT EXISTS` patterns that re-scan large data sets. Overall, AI demonstrated strong structural SQL knowledge but lacked awareness of execution cost and data semantics. AI-generated SQL should not be trusted without review in scenarios involving complex joins, composite keys, or multi-level aggregation. In this schema, queries involving `takes`, `section`, and `teaches` are especially risky because failing to join on all four key attributes (course_id, sec_id, semester, year) can produce incorrect results through row multiplication. Similarly, queries combining counts and averages are prone to subtle errors if AI does not account for differences in data granularity. In a professional workflow, I would use AI-generated SQL as a starting point, but not a final solution. AI is useful for quickly producing a draft query or exploring different approaches, but every query must go through a review process. This would include verifying join conditions, aggregation logic and using (EXPLAIN ANALYZE) to review execution plans. This lab significantly improved my understanding of how SQL queries actually execute. Before, I focused mainly on producing correct results, but I now understand that query structure directly affects performance and correctness.For example, I learned that joining tables before aggregation can unintentionally duplicate data, and that functions like COUNT and AVG depend heavily on the level of detail in the data. Moving forward, I will approach SQL more deliberately by verifying joins against true primary keys, considering performance implications early, and using tools like (EXPLAIN ANALYZE) to validate efficiency. This matters because in real-world systems, even small logical mistakes or inefficiencies can scale into significant correctness issues or performance bottlenecks.